Lab Manual for Data Structure and Algorithm (LAB-03)

Implementation of Array

Lab Instructor: Muniba khan:

Goal for Today's Lab

- **Understand and apply looping structures, arrays, rand() function, and functions in C++.**

Topic 1: Arrays in C++

An **array** is a collection of elements of the same type stored in **contiguous memory locations**.

- Each element is accessed using its **index**, starting from 0.
- Arrays make it easier to store and process large sets of data efficiently.

Syntax:

```
int arr[5];                 // Declaration
int arr[] = {1, 2, 3, 4, 5};    // Initialization
```

Example:

```
int marks [5] = {85, 90, 78, 88, 76};
```

Key Concepts

- **Array Element:** Each item stored in the array.
- **Array Index:** The position of each element (starts from 0).
- **Traversal:** Accessing each element of the array using a loop.

Example 1: Random Number Array

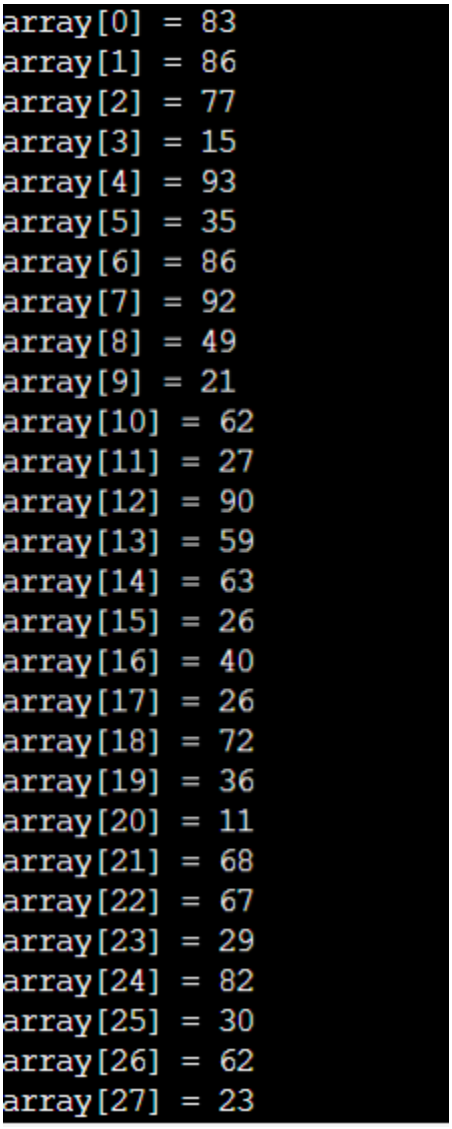
Problem:

Create an integer array of size 100 and initialize it with random numbers between 0–99. Print all elements.

```
#include<iostream>

#include<stdlib.h>  // for rand()

using namespace std;


int main() {

   int array[100];

   for (int i = 0; i < 100; i++) {

      array[i] = rand() % 100;  // random number 0–99

   }


   for (int i = 0; i < 100; i++) {

      cout << "array[" << i << "] = " << array[i] << endl;

   }

   return 0;

}
```

```
#include<iostream>
#include<stdlib.h>  // for rand()
using namespace std;

int main() {
    int array[100];
    for (int i = 0; i < 100; i++) {
        array[i] = rand() % 100;  // random number 0-99
    }

    for (int i = 0; i < 100; i++) {
        cout << "array[" << i << "] = " << array[i] << endl;
    }
    return 0;
}
```

Output:



Note: You can use srand(time(0)); before the loop (after including <ctime>) to get different random numbers every time you run the program**.**

Traversal of Array: Visiting each array element one by one, often using a loop.

```
for (int i = 0; i < 100; i++) {
    cout << "array[" << i << "] = " << array[i] << endl;
}
```

Example 2: Calculate Sum, Average, Maximum, and Minimum of Array Elements

Problem:

Write a program to:

- *Read n integers into an array.*
- *Calculate and display the **sum**, **average**, **maximum**, and **minimum** values.*

```
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements (max 100): ";
    cin >> n;

    int arr[100];

    // Input elements into array
    cout << "Enter " << n << " integers: " << endl;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Initialize variables
    int sum = 0;
    int max = arr[0];
    int min = arr[0];

    // Traverse array to find sum, max, min
    for (int i = 0; i < n; i++) {
        sum += arr[i];
        if (arr[i] > max)
            max = arr[i];
        if (arr[i] < min)
            min = arr[i];
    }

    // Calculate average
    float average = (float)sum / n;
```

```cpp
// Calculate average
float average = (float)sum / n;

// Display results
cout << "\nSum = " << sum << endl;
cout << "Average = " << average << endl;
cout << "Maximum = " << max << endl;
cout << "Minimum = " << min << endl;

return 0;
```

```
Enter number of elements (max 100): 10
Enter 10 integers:
56
34
23
1
45
8
9
6
54
3

Sum = 239
Average = 23.9
Maximum = 56
Minimum = 1
```

LAB Activity Tasks:

Types of Arrays:

1. **1D Array (One-dimensional):**
    - Elements are stored in a single row.
    - **Indexing:** Starts from 0 (lower bound) to n-1 (upper bound), where n = size of the array.
    - **Size:** Total number of elements.

1D Array Activity: Problem Statement:

Write a program to store 5 numbers in an array and print the 3rd element.

```c
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};  // 1D array
    printf("3rd element is: %d\n", arr[2]);  // Accessing element at index 2
    return 0;
}
```

```
3rd element is: 30


...Program finished with exit code 0
Press ENTER to exit console.
```

2. **2D Array (Two-dimensional):**

- Elements are stored in rows and columns (like a table).
- **Indexing:** arr[i][j] → i = row index, j = column index
- **Size:** Total elements = rows × columns
- **Lower bound:** 0 for both row and column
- **Upper bound:** rows-1 for row, columns-1 for column

Problem Statement:

Write a program to store numbers in a 2×3 matrix and print the element in the 2nd row, 3rd column.

```c
#include <stdio.h>

int main() {
    int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };  // 2D array
    printf("Element at 2nd row, 3rd column is: %d\n", arr[1][2]);
    return 0;
}
```

```
Element at 2nd row, 3rd column is: 6
```

3. Topic: Loops in C++

Loops are used to execute a block of code repeatedly until specific condition is true.

Loop Type	Description	Example Range
for	Used when number of iterations is known.	1 to 10
while	Used when condition is checked before loop body.	1 to 10
do-while	Used when body must run at least once.	1 to 10
continue	Skips the current iteration and moves to the next one.	Used to skip odd numbers

(a) For Loop

```
for (int i = 1; i <= 5; i++) {
cout << i << " ";
}
```

(b) While Loop

```
int i = 1;
while (i <= 5) {
cout << i << " ";
i++;
}
```

(c) Do-While Loop

int i = 1;

do {

cout << i << " ";

i++;

} while (i <= 5);

Lab Activity 5:

Problem Statement:
Write a C++ program that uses **all three types of loops** (for, while, and do-while) to print numbers from **1 to 10**.
Then modify the code to print **only even numbers** using the continue statement.

Code;

```cpp
#include <iostream>
using namespace std;

int main() {

    cout << "Using FOR loop:" << endl;
    for (int i = 1; i <= 10; i++) {
        cout << i << " ";
    }

    cout << "\n\nUsing WHILE loop:" << endl;
    int j = 1;
    while (j <= 10) {
        cout << j << " ";
        j++;
    }

    cout << "\n\nUsing DO-WHILE loop:" << endl;
    int k = 1;
    do {
        cout << k << " ";
        k++;
    } while (k <= 10);

    // PART 2: Using continue to print only even numbers
    cout << "\n\nPrinting only EVEN numbers using FOR loop and continue:"
    for (int i = 1; i <= 10; i++) {
        if (i % 2 != 0)
            continue;   // skip odd numbers
        cout << i << " ";
    }

    return 0;
}
```

```
Using FOR loop:
1 2 3 4 5 6 7 8 9 10

Using WHILE loop:
1 2 3 4 5 6 7 8 9 10

Using DO-WHILE loop:
1 2 3 4 5 6 7 8 9 10

Printing only EVEN numbers using FOR loop and continue:
2 4 6 8 10
```

Topic 3: Functions in C++

A **function** is a block of code designed to perform a specific task. Functions make code **modular**, **readable**, and **reusable**.

Types of Functions

1. **Void functions:** Do not return a value.
   Example:

```
void printHello() {
    cout << "Hello!";
}
```

Value-returning functions: Return a value to the caller.
Example:

```
int add(int a, int b) {
    return a + b;
}
```

Function Components

1. **Prototype:** Declaration of the function before main().
    - Example: int add(int, int);

2. **Definition:** Actual code of the function.
3. **Call:** Using the function in main()

Example 2: Number Classification

Problem:
Write a program that reads 10 numbers and counts how many are even, odd, or zero using functions.

function prototypes:

```
void initialize(int& zeroCount, int& oddCount, int& evenCount);

void getNumber(int& num);

void classifyNumber(int num, int& zeroCount, int& oddCount, int& evenCount);

void printResults(int zeroCount, int oddCount, int evenCount);
```

What Is a Function Prototype?

A **function prototype** is the **declaration** of a function — it tells the compiler:

- The **name** of the function
- The **type of data it returns**
- The **types and order of its parameters**

It comes **before main()**, so the compiler knows how to call that function later.

What Are Function Definitions?

A **function definition** is where the **actual code (body)** of the function is written — what the function *does*.

Each definition includes:

1. Return type (e.g., void, int, float, etc.)

2. Function name
3. Parameter list (if any)
4. The code block { ... }

```
void initialize(int& zeroCount, int& oddCount, int& evenCount) {
    zeroCount = oddCount = evenCount = 0;
}

void getNumber(int& num) {
    cin >> num;
}

void classifyNumber(int num, int& zeroCount, int& oddCount, int&
evenCount) {
    if (num == 0) zeroCount++;
    else if (num % 2 == 0) evenCount++;
    else oddCount++;
}

void printResults(int zeroCount, int oddCount, int evenCount) {
    cout << "There are " << evenCount << " even, "
        << oddCount << " odd, and "
        << zeroCount << " zeros in the list." << endl;
}
```

Function Calls (in main())

Inside main(), you are **calling** these functions:

```
initialize(zeros, odds, evens);                    // Step 1: call initialize()
getNumber(number);                                  // Step 2: call getNumber()
classifyNumber(number, zeros, odds, evens); // Step 3: call classifyNumber()
printResults(zeros, odds, evens);                          // Step 4: call printResults()
```

Example 2. This problem is typical Number classification system. In the simplest form when the program starts it prompts the user to enter some numbers. The User enters space separated numbers. The program then responds by printing number of even, odd and zeros in the list entered by user.

Please enter 10 number : **23 23 12 0 43 23 3 2 43 56**

There are 3 even 7 odd and 1 zeros in the list

- **Algorithm**

    i) Initialize the variables, zeros, odds, and evens to 0.

    ii) Read a number.

    iii) If the number is even, increment the even count, and if the number is zero, increment the zero count; otherwise, increment the odd count.

    iv) Repeat Steps 2 and 3 for each number in the list.

Write the following four functions to solve the number classfication problem

void initialize(int& zeroCount, int& oddCount, int& evenCount);

void getNumber(int& num);

void classifyNumber(int num,int& zeroCount, int& oddCount, int& evenCount);

void printResults(int zeroCount,int oddCount,int evenCount);

```
//*************************************************************
// DSA Lab: Functions in C++
// Program: Number Classification
// Description: Reads numbers and counts how many are even, odd, or zero
//*************************************************************

#include<iostream>
using namespace std;

// Constant for total numbers to be read
const int N = 10;

// ---------------- Function Prototypes ----------------
void initialize(int& zeroCount, int& oddCount, int& evenCount);
void getNumber(int& num);
void classifyNumber(int num, int& zeroCount, int& oddCount, int& evenCount);
void printResults(int zeroCount, int oddCount, int evenCount);
```

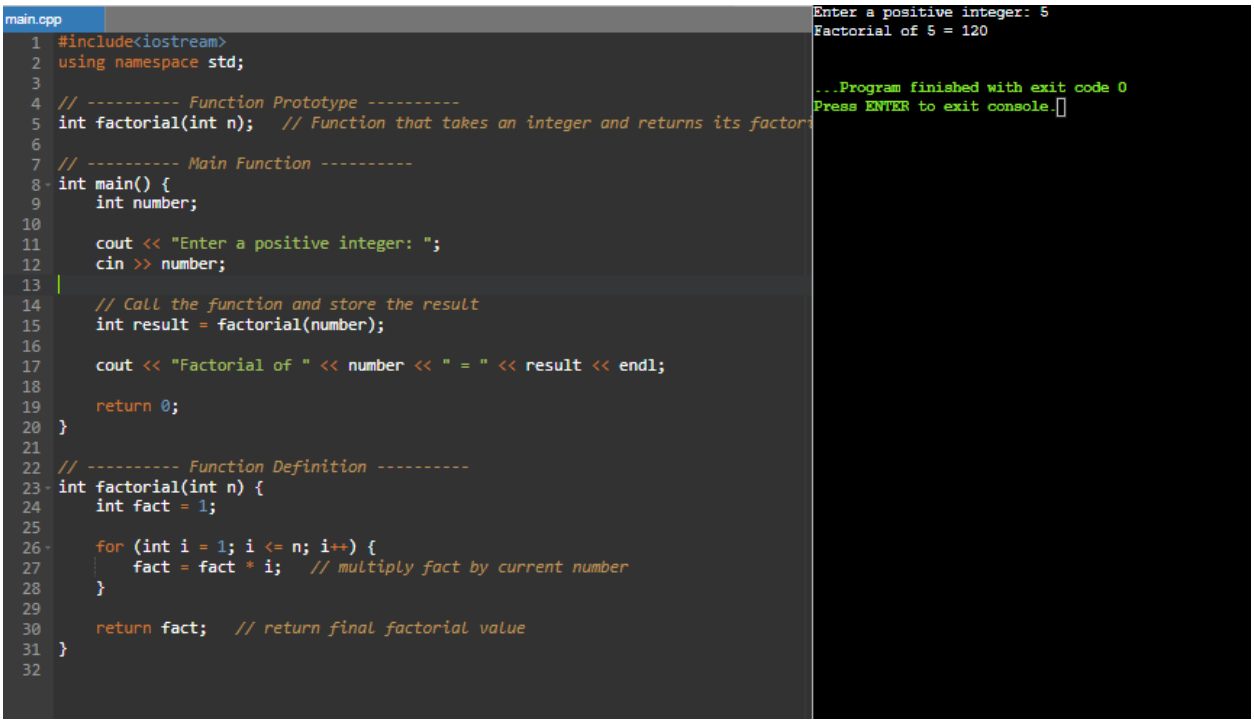
```
// ---------------- Main Function ----------------
int main() {
    int zeros, odds, evens;   // Counters
    int number;               // Variable to store each input

    // Step 1: Initialize counters
    initialize(zeros, odds, evens);

    cout << "Please enter " << N << " integers:" << endl;

    // Step 2: Loop to input and classify numbers
    for (int i = 0; i < N; i++) {
        getNumber(number);  // Get input from user
        classifyNumber(number, zeros, odds, evens);  // Classify each number
    }

    // Step 3: Display final results
    printResults(zeros, odds, evens);

    return 0;
}

// ---------------- Function Definitions ----------------

// Function to initialize counters
void initialize(int& zeroCount, int& oddCount, int& evenCount) {
    zeroCount = 0;
    oddCount = 0;
    evenCount = 0;
}

// Function to input a single number
void getNumber(int& num) {
    cin >> num;
}

// Function to classify a number as even, odd, or zero
void classifyNumber(int num, int& zeroCount, int& oddCount, int& evenCount)
{
    if (num == 0) {
```

```
        zeroCount++;
    }
    else if (num % 2 == 0) {
        evenCount++;
    }
    else {
        oddCount++;
    }
}
// Function to print the results
void printResults(int zeroCount, int oddCount, int evenCount) {
    cout << "\n---------- Classification Results ----------" << endl;
    cout << "Even numbers: " << evenCount << endl;
    cout << "Odd numbers:  " << oddCount << endl;
    cout << "Zeros:        " << zeroCount << endl;
    cout << "-------------------------------------------" << endl;
}
```

Example 3: Calculate the Factorial of a Number Using a Function

Problem Statement

Write a program that calculates the **factorial** of a given number using a **user-defined function**.



Problem Statement:

Write a C++ program that **simulates rolling a dice 10 times** using the rand() function. The program should display the result of each roll (numbers between 1 and 6).

```cpp
#include <iostream>
#include <cstdlib>   // For rand() and srand()
#include <ctime>     // For time()
using namespace std;

int main() {
    // Seed the random number generator with current time
    srand(time(0));

    cout << "Rolling a dice 10 times:\n";

    for (int i = 1; i <= 10; i++) {
        int roll = rand() % 6 + 1;  // Random number between 1
        cout << "Roll " << i << ": " << roll << endl;
    }

    return 0;
}
```

```
Rolling a dice 10 times:
Roll 1: 5
Roll 2: 1
Roll 3: 2
Roll 4: 3
Roll 5: 3
Roll 6: 6
Roll 7: 1
Roll 8: 6
Roll 9: 3
Roll 10: 6
```

Definition of sizeof() in C++

Definition:
The sizeof() operator in C++ is used to find the **memory size (in bytes)** of any data type, variable, array, or object.
It tells us how much space a particular type or value takes up in memory.

sizeof(data_type)

sizeof(variable_name)

Example: Basic use of sizeof()

```
#include <iostream>

using namespace std;


int main() {
    int a;
    double b;
    char c;
    float d;


    cout << "Size of int: " << sizeof(a) << " bytes" << endl;
    cout << "Size of double: " << sizeof(b) << " bytes" << endl;
    cout << "Size of char: " << sizeof(c) << " bytes" << endl;
    cout << "Size of float: " << sizeof(d) << " bytes" << endl;


    return 0;
}
```

Lab Assignment Tasks:

Question 1 – 1D Array

Write a C++ program that stores **10 integers** in a 1D array.
Your program should:

1. Display all the numbers entered.
2. Print all **even numbers**.

Question 2 – 2D Array

Write a C++ program that inputs a **3×3 matrix** and:

1. Displays the matrix in table form.
2. Calculates and prints the **sum of each row** and **each column**.
3. Displays the **main diagonal elements**.

Question 3 – Loops (for, while, do-while)

Write a C++ program to print the **multiplication table** of a number entered by the user using:

1. for loop
2. while loop
3. do-while loop
   Show results of all three loops **separately**.

Question 4 – Functions

Write a C++ program using **functions** to count how many numbers in a list of **10 integers** are **even**, **odd**, or **zero**.
Use separate functions for:

- Taking input

- Classifying numbers
- Displaying results

Question 5 – 1D Array (Reversal)

Write a C++ program to **reverse a 1D array** of n integers and print the reversed array.
Example:
Input → 1 2 3 4
Output → 4 3 2 1

Task 6 – Maximum, Minimum & Difference

Problem Statement:
Write a C++ program that takes **5 integers** from the user and stores them in a 1D array.

Your program should:

1. Display all the numbers entered.
2. Find and display the **maximum number**.
3. Find and display the **minimum number**.
4. Calculate and display the **difference between the maximum and minimum** numbers.

Task 7. Problem Statement:

Write a program to calculate the factorial of a number using a function.

Task 8. Problem Statement:

Write a C++ program that asks the user to enter the number of elements for an array of type int. Then:

1. Create the array and display its total size in bytes using sizeof().
2. Calculate how many elements of that type can fit in **100 bytes**.

Note: Report should include all lab activities assigned in lab &Lab task fully formatted report with descriptions of each code you execute.

Deadline: 30 October,2025

----------**END**----------